

MÓDULO 6

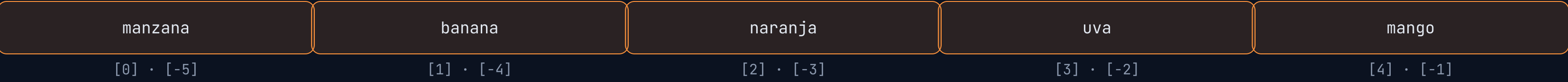
Listas y Tuplas

Sesión 6 · 3 horas — objetivo: manejar listas (índices, slicing, métodos, enumerate, anidadas) y tuplas inmutables.

¿Qué es una lista?

Colección ordenada y mutable de elementos. Puede contener cualquier tipo de dato.

```
frutas = ['manzana', 'banana', 'naranja', 'uva', 'mango']
```





Ordenada
Los elementos mantienen su posición e índice



Mutable
Se puede modificar: agregar, eliminar, cambiar



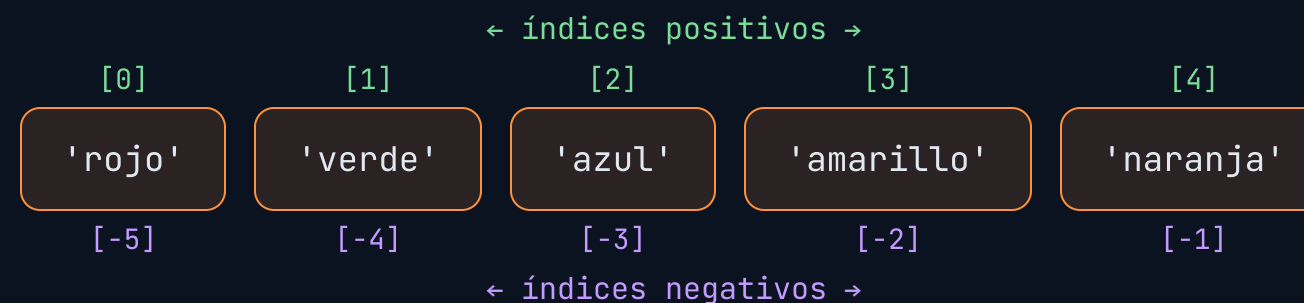
Heterogénea
Mezcla diferentes tipos de datos



Indexada
Primer elemento en índice 0, último en -1

Acceso por índice – positivos y negativos

Los positivos van de izquierda a derecha, los negativos de derecha a izquierda.



```
colores = ['rojo', 'verde', 'azul', 'amarillo', 'naranja']
print(colores[0])    # 'rojo' ← primero
print(colores[-1])   # 'naranja' ← último
print(colores[2])    # 'azul'
print(colores[-2])   # 'amarillo'
```

```
# Modificar un elemento
colores[1] = 'morado'
print(colores)
# ['rojo', 'morado', 'azul', ...]

# len() – cuántos elementos
print(len(colores)) # 5
```

⚠ colores[10] en lista de 5 → IndexError. Verifica con len() antes de acceder a un índice.

Slicing – obtener sublistas

lista[inicio:fin:paso] – el elemento en 'fin' NO se incluye.

nums = [10, 20, 30, 40, 50, 60, 70, 80] ← lista de referencia

nums[2:5]

[30, 40, 50]

Índices 2, 3, 4

nums[:4]

[10, 20, 30, 40]

Desde inicio hasta 3

nums[5:]

[60, 70, 80]

Desde índice 5 al fin

nums[::2]

[10, 30, 50, 70]

Cada 2 elementos

nums[::-1]

[80, 70 ... 10]

Invertida (paso -1)

nums[1:7:3]

[20, 50]

Del 1 al 6, de 3 en 3

Métodos principales de lista

Se llaman con `lista.metodo()`. Los más importantes:

`append(x)`

Agrega x al final

`insert(i, x)`

Inserta x en posición i

`remove(x)`

Elimina 1ª ocurrencia de x

`pop(i)`

Elimina y retorna elemento i

`sort()`

Ordena la lista (in place)

`reverse()`

Invierte la lista (in place)

`index(x)`

Retorna índice de x

`count(x)`

Cuenta ocurrencias de x

`copy()`

Copia superficial

`extend(l)`

Agrega todos los elem. de l



`sort()` modifica la lista original. Para conservarla, usa `sorted(lista)` que retorna una NUEVA lista ordenada.

Recorrer con for y enumerate()

enumerate() retorna pares (índice, elemento) — la forma idiomática de recorrer listas.

```
# Sin enumerate – verboso
frutas = ['manzana', 'banana', 'naranja']
for i in range(len(frutas)):
    print(f"{i}: {frutas[i]}")
```

```
# Con enumerate – Pythónico ✅
frutas = ['manzana', 'banana', 'naranja']
for i, fruta in enumerate(frutas, start=1):
    print(f"{i}: {fruta}")
```

```
# Acumular resultados con for
precios = [150.0, 45.5, 320.0, 89.0, 210.0]
total=0 caros=0 baratos=[]
for i, precio in enumerate(precios, 1):
    total += precio
    if precio > 100: caros += 1
    else: baratos.append(precio)
print(f"Total: Bs.{total:.2f} Caros: {caros} Baratos: {baratos}")
```

Listas anidadas – matrices

Una lista puede contener otras listas. Se accede con dos índices: `matriz[filas][columna]`.

```
matriz = [[1,2,3],[4,5,6],[7,8,9]]
```

1 [0][0]	2 [0][1]	3 [0][2]
4 [1][0]	5 [1][1]	6 [1][2]
7 [2][0]	8 [2][1]	9 [2][2]

■ Fila 0 ■ Fila 1 ■ Fila 2

```
# Acceder: matriz[filas][columna]
print(matriz[0][0])    # 1
print(matriz[1][2])    # 6
print(matriz[2][1])    # 8
```

```
# Recorrer con for anidado
for fila in matriz:
    for elem in fila:
        print(elem, end=" ")
    print() # salto de línea
```

```
# Salida: 1 2 3 / 4 5 6 / 7 8 9
```

Tuplas – colecciones inmutables

Como una lista pero inmutable: una vez creada, sus elementos NO pueden modificarse.

📄 Lista [] – MUTABLE

```
carrito = ['Laptop', 'Mouse']
carrito.append('Teclado') # ✅
carrito[0] = 'PC'         # ✅
carrito.remove('Mouse')   # ✅
```

🚫 Tupla () – IMMUTABLE

```
coords = (19.04, -65.26)
coords.append(0) # ❌ AttributeError
coords[0] = 20   # ❌ TypeError
# ✅ Solo lectura, indexing, slicing
```

```
# Cuando usar cada una
notas = [85, 92, 78] # Lista – notas cambian durante el semestre
coordenadas = (19.04, -65.26) # Tupla – GPS no cambia
colores_rgb = (255, 128, 0) # Tupla – color fijo
x, y = (10, 20) # Desempaquetado de tupla
```

💡 Para tupla de 1 elemento: (42,) – la coma es OBLIGATORIA. Sin ella, (42) es solo un número.

Ejercicio integrador — Sistema de notas

 Combinar listas, tuplas, slicing, métodos y enumerate() en un programa real

```
# Tupla – datos fijos del curso
CURSO = ("Python Básico", "2024")
# Listas – datos que cambian
alumnos = ["Ana", "Carlos", "María", "Luis"]
notas = [85, 62, 91, 74]
alumnos.append("Sofia"); notas.append(88)
# Ranking (zip + sort)
pares = list(zip(notas, alumnos))
pares.sort(reverse=True)
print(f"=== {CURSO[0]} – Ranking ===")
for pos, (nota, nombre) in enumerate(pares, 1):
    est = 'Aprobado' if nota >= 51 else 'Reprobado'
    print(f"{pos}. {nombre:<10} {nota}/100 {est}")
solo_notas = [p[0] for p in pares]
print(f"Top 3: {solo_notas[:3]}")
```



Tupla

CURSO – dato fijo



Listas

alumnos + notas



append()

nuevo estudiante



zip()

combina 2 listas



enumerate

ranking numerado



Slicing

top 3 con [:3]

✓ LO QUE APRENDISTE HOY

Módulo 6 en resumen



Listas []

Colección ordenada y mutable



Índices

Positivos [0..n] y negativos [-n..-1]



Slicing

lista[inicio:fin:paso]



Métodos

append, insert, remove, pop, sort...



enumerate()

Índice + valor en el mismo for



Anidadas

Matrices: lista[filas][columnas]



Tuplas ()

Inmutables — para datos fijos



Desempaquetado

x, y = (10, 20)



Próxima sesión: **Módulo 7 — Diccionarios y Conjuntos**